



Technische
Universität
Braunschweig



10-Sept-24

igp



Exploring MLRSNet: A Game – Changer in Remote Sensing

Disha Ghosh & Nizaanth Raja

MSc. Data Science & MSc. Computational Science

Deep Learning Course, IGP-TUBS, Dr. -Ing. Mehdi Maboudi, Dr. Pedro Achanccaray

Welcome to our presentation

Objective: To introduce and explore the MLRSNet dataset, while comparing the performance of the EfficientNet B0 base model (with pre-trained weights) to other models using pre-trained weights as mentioned in the original paper. Additionally, this work also involves training the EfficientNet B0 model from scratch for further comparison.

Outline

1. Brief Overview of the MLRSNet Dataset

- Introduction to the MLRSNet dataset and its significance in remote sensing image classification.

2. Exploratory Data Analysis (EDA)

- Visualizations for images per category / label.

3. EfficientNet Model

- Architecture and problem formulation.

4. Experimental Setup

- Experimental Protocols (train / validation / test distribution, pre-trained & training from scratch, metrics, etc.)
- Hyperparameters (Package, Batch size, learning rate, epochs, optimizer)

5. Results and Visualizations

- Model performance and comparison with state – of – art models mentioned in the paper.

6. Summary

- Key findings and insights.
- Limitations and potential enhancements.

MLRSNet Dataset Overview

1. **Purpose:** Designed for multi-label classification of remote sensing images
2. **Total Images:** 109,161 high spatial resolution optical satellite images
3. **Image Size:** 256×256 pixels
4. **Categories:** 46 categories, including both natural and man-made objects
 - Sample Size per Category: 1,500 to 3,000 images
5. **Labels:** 60 predefined class labels
 - Labels per Image: Ranges from 1 to 13
6. **Geographical Diversity:** Images represent various geographical locations and environmental conditions, which helps in training models that generalize well across different regions.
7. **Use Case:** Enhances the development of robust machine learning models for remote sensing applications by providing a diverse and comprehensive dataset, which helps in pushing the boundaries of machine learning models in terms of accuracy and generalization.

Difference Between Categories and Labels

Labels	Categories
1. 'Labels' are referred to the specific labels assigned to each image in the dataset, indicating the presence of certain features or characteristics within that image. There are 60 pre-defined labels .	1. "Categories" refer to the different types of scenes or land-use and land-cover types that are represented in the dataset. There are 46 distinct categories in total.
2. These labels include cloud, desert, farmland, intersection and buildings .	2. These categories include various broad scene types like Residential, Industrial, Farmland, Forest, Water, Airport , and Highway .

EDA – Understanding Data

Exploratory Data Analysis (EDA) provided valuable insights into the characteristics of the MLRSNet dataset.

The initial analysis involved summarizing the dataset's statistical properties, including the distribution of images across various categories and the frequency of occurrence for each label.

Source: [Exploratory Data Analysis_MLRSNet](#)

EDA Visualisations



Fig.1. Labels and Categories for each image

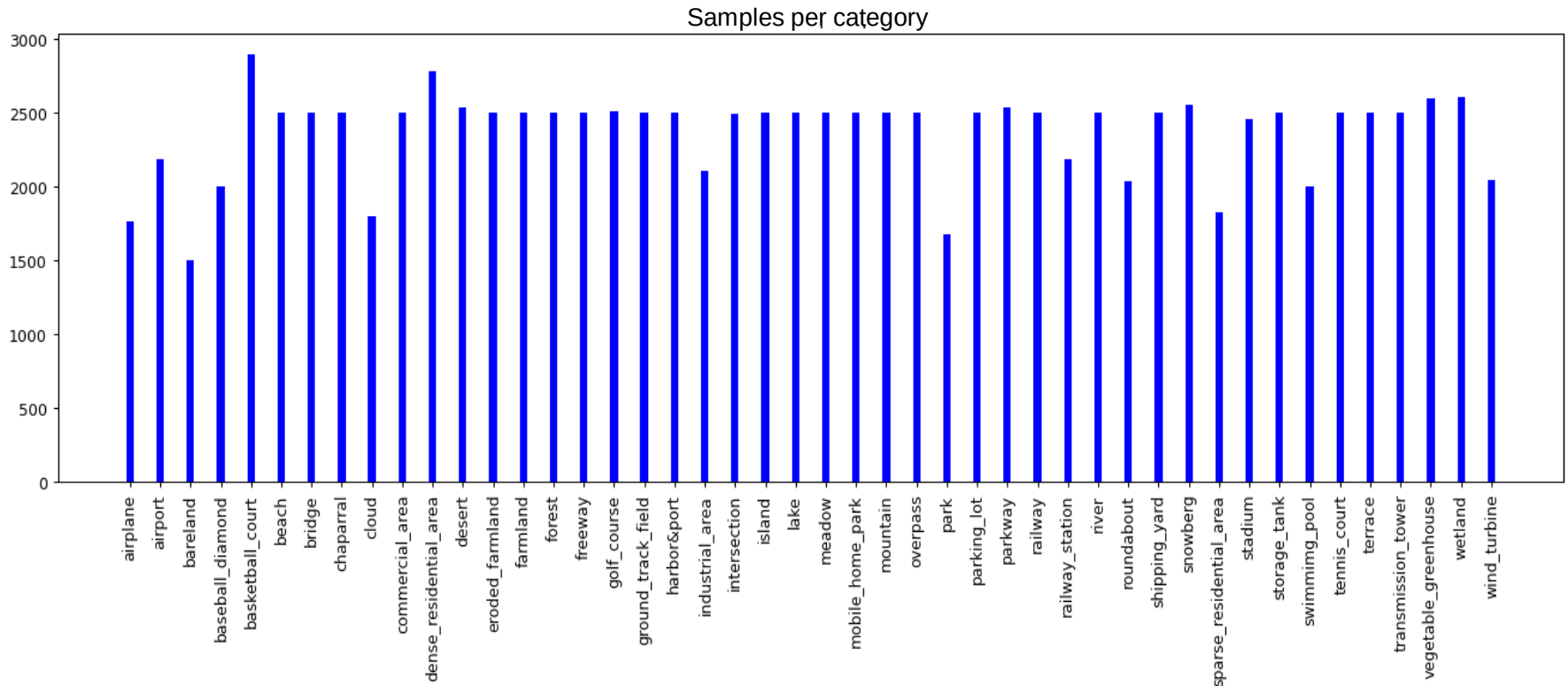


Fig.2. Distribution of Number of samples per category

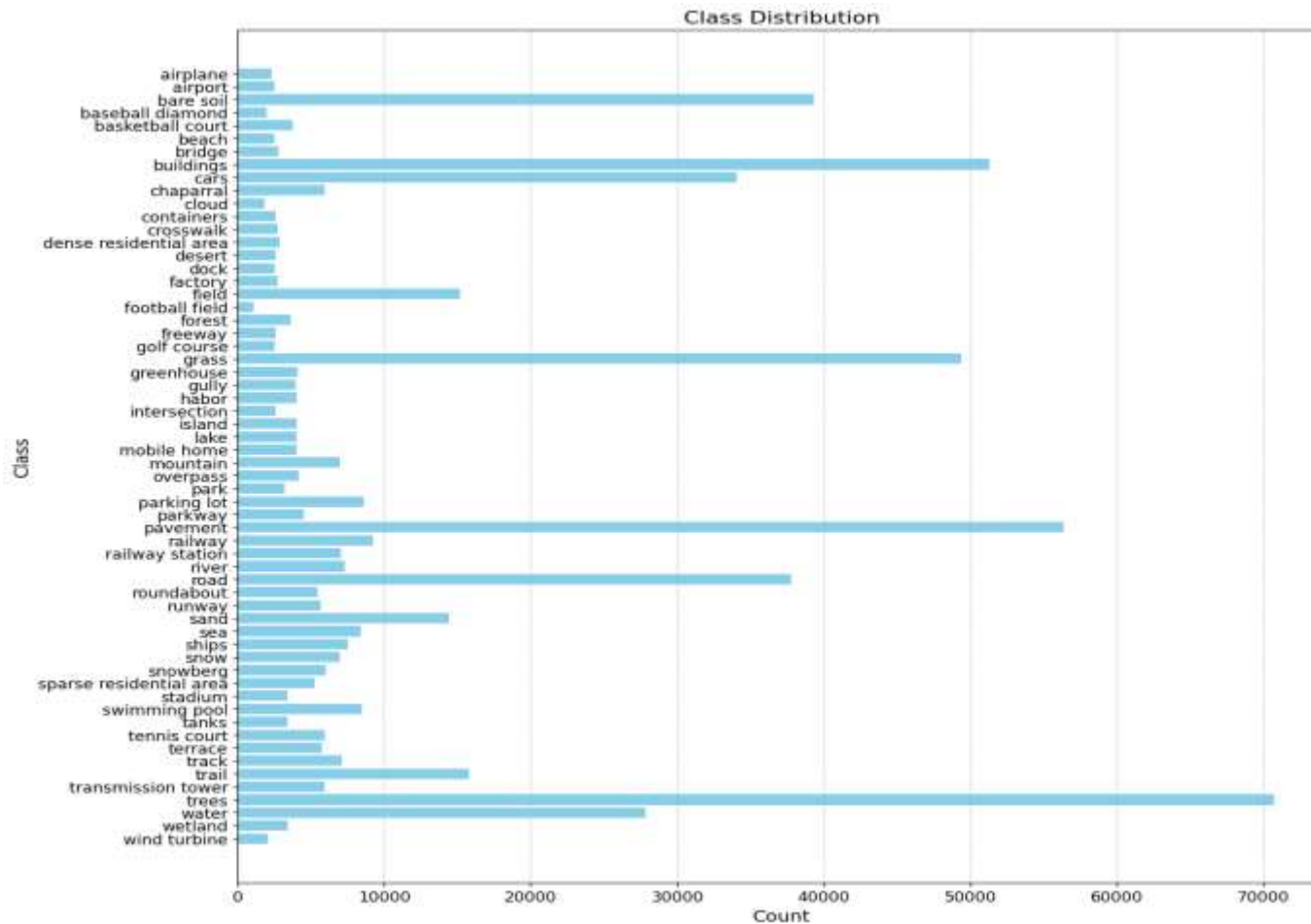


Fig.3. Distribution of Number of samples per Label

Key Concepts in EfficientNets

- **Depth (D):** The depth of a neural network refers to the number of layers, or more specifically, the number of convolutional layers or blocks in the network.
- **Width (W):** The width of a neural network refers to the number of channels (also known as filters or feature maps) in a convolutional layer. In other words, it represents how many neurons (or units) are there in each layer.
- **Resolution (R):** The resolution of a network refers to the input image size. Higher resolution means the network processes larger images, which contain more details.

The challenges associated with scaling either of these are as follows:

- Increasing depth can lead to vanishing gradients and overfitting.
- Increasing width can cause model redundancy and excessive computational demand.
- Increasing resolution requires significantly more memory and processing power.

EfficientNet's Novel Approach - Compound Scaling

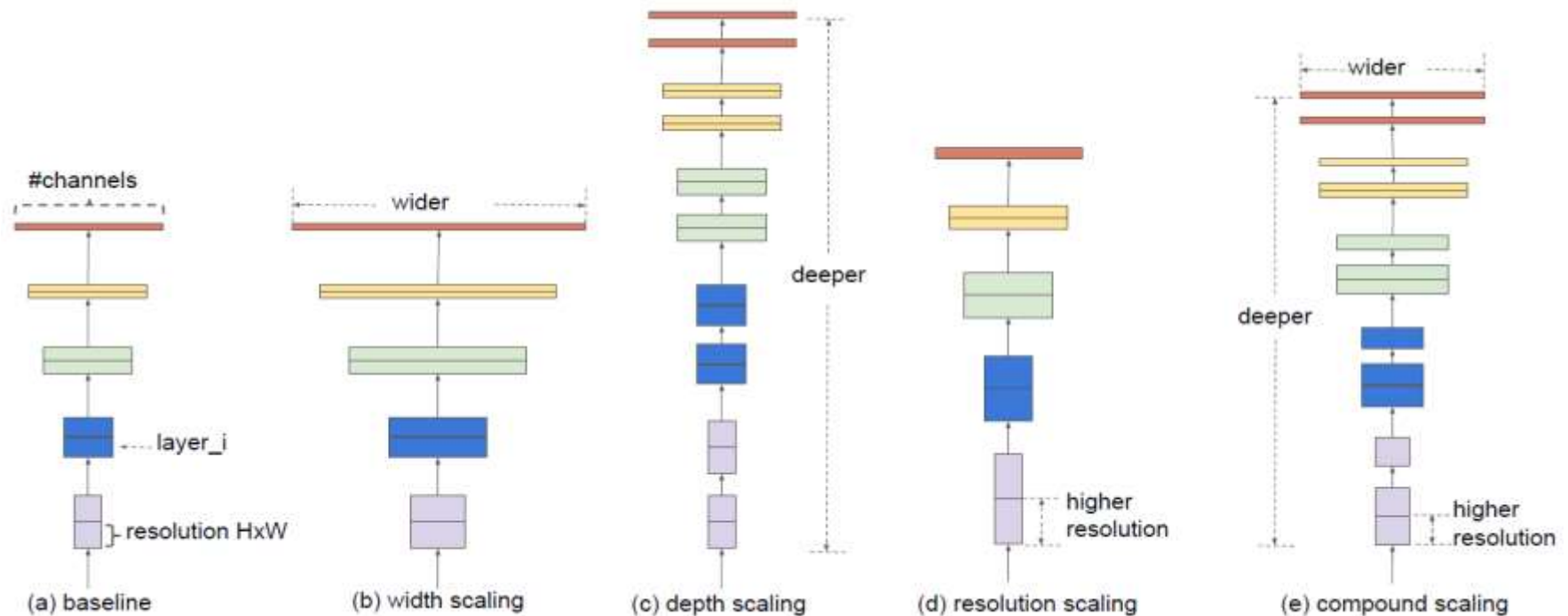


Fig.4. **Model Scaling Visualisation.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Source: [Original Paper](#)

Problem Formulation for the Proposed Compound Scaling Method

- **Goal:** Expand network depth, width, and resolution without changing F_i predefined in the baseline network.
- **Uniform Scaling:** All layers are scaled by a constant ratio to simplify adjustments.
- **Scaling Factors:**
 - **Depth (L_i):** $d = \alpha\varphi$
 - **Width (C_i):** $w = \beta\varphi$
 - **Resolution (H_i, W_i):** $r = \gamma\varphi$
- **Design Rule:** $\alpha \times \beta^2 \times \gamma^2 \approx 2$, ensuring balanced scaling.
- φ : User-defined coefficient for uniform scaling across depth, width, and resolution.

Architecture

- The evaluation of the proposed scaling method was achieved using existing ConvNets but also develop a new mobile-size baseline called EfficientNet to better demonstrate its effectiveness.
- Inspired by [Tan et al. \(2019\)](#) a baseline network (EfficientNet – B0) was developed using a multi-objective Neural Architecture Search (NAS) that optimizes both accuracy and FLOPS.
- We use the same search space as Tan et al. (2019) and define the optimization goal as:

$$ACC(m) [FLOPS(m) / T]^w$$

where $ACC(m)$ and $FLOPS(m)$ denote the accuracy and FLOPS of model m , T is the target FLOPS, and $w = -0.07$ is a hyperparameter controlling the trade-off between accuracy and FLOPS.

- The building block of this architecture is the [mobile inverted bottleneck architecture](#) MBConv (originally introduced in the MobileNetV2, a design reduces the number of parameters and computational costs while still delivering strong performance)), to which there is an addition of [squeeze – and – excitation optimisation](#). (enhance the network's ability to focus on important features by adaptively recalibrating channel-wise feature responses).

Architecture

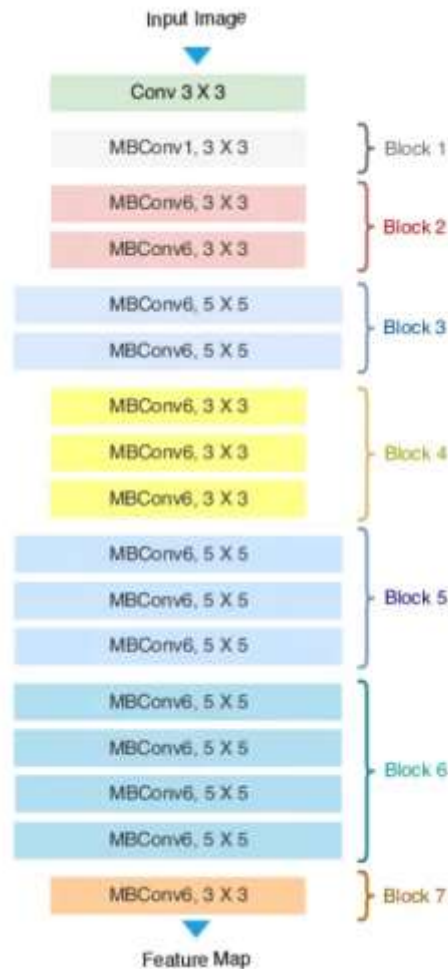


Fig.5.EfficientNet Architecture
[Have a look at the paper here!](#)

Here,

- kxk: kernel size.
- MBConv: Expansion Ratio for channels.
- Blocks: Groupings of layers that progressively deepen the network and extract more abstract and complex features from the image.

Experimental Setup

Setup	Training	Validation	Testing	Pre - trained Weights
Split 1	20%	10%	70%	Yes (ImageNet)
Split 2	30%	10%	60%	Yes (ImageNet)
Split 3	40%	10%	50%	Yes (ImageNet)
Scratch Model	70%	10%	20%	No

Table 1. Model Configurations

Experimental Setup

Hyperparameter Tuning

- Package: Keras
- Batch Size: 32
- Learning Rate: 0.01
- Epochs: 10
- Optimizer: Adam

Metrics

- F1 Score (Samples): Evaluates the balance between precision and recall for multi-label classification.
- Mean Average Precision (mAP): Measures the precision across all class labels.

Results

CNNs	20%	30%	40%
MLRSNet-InceptionV3	81.50	82.33	84.84
MLRSNet-VGGNet16	67.88	72.66	75.39
MLRSNet-VGGNet19	66.12	69.53	73.60
MLRSNet-ResNet50	82.65	84.28	86.01
MLRSNet-ResNet101	83.26	84.19	85.72
MLRSNet-DenseNet121	75.96	77.99	80.25
MLRSNet-DenseNet169	82.16	86.42	87.35
MLRSNet-DenseNet201	87.25	87.84	88.77
MLRSNet-EfficientNetB0	93.10	94.10	94.58

Table 2. Mean Average Precision (mAP): Comparison of our model with the other eight fine tuned model mentioned in the paper

Results

CNNs	20%	30%	40%
MLRSNet-InceptionV3	0.7746	0.8016	0.8146
MLRSNet-VGGNet16	0.5743	0.6534	0.6855
MLRSNet-VGGNet19	0.5677	0.6120	0.6329
MLRSNet-ResNet50	0.7530	0.8176	0.8353
MLRSNet-ResNet101	0.7618	0.7703	0.8226
MLRSNet-DenseNet121	0.7154	0.7389	0.7571
0MLRSNet-DenseNet169	0.8138	0.8408	0.8521
MLRSNet-DenseNet201	0.8381	0.8414	0.8538
MLRSNet-EfficientNetB0	0.8355	0.8492	0.8539

Table 2. F1 Score (Samples): Comparison of our model with the other eight fine tuned model mentioned in the paper

Visualizations for Base model for the split ratio 20-10-70.

Image 1
True: chaparral, cloud, golf_course, snowberg
Pred: chaparral, golf_course, snowberg



Image 2
True: chaparral, lake, snowberg, swimming_pool
Pred: bareland, chaparral, lake, snowberg, swimming_pool



Image 3
True: chaparral, cloud, snowberg, swimming_pool
Pred: bareland, chaparral, lake, snowberg, swimming_pool

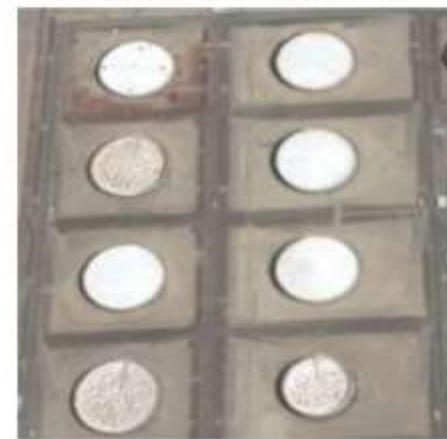


Image 4
True: bareland, lake, parking_lot
Pred: bareland, lake, parking_lot



Image 5
True: bareland, chaparral, cloud, lake, snowberg, swimming_pool
Pred: bareland, chaparral, cloud, lake, snowberg, swimming_pool



Visualizations for Base model for the split ratio 30-10-60.

Image 1
True: chaparral, cloud, golf_course, snowberg
Pred: bareland, chaparral, golf_course, snowberg



Image 2
True: chaparral, lake, snowberg, swimming_pool
Pred: bareland, chaparral, lake, snowberg, swimming_pool



Image 3
True: chaparral, cloud, snowberg, swimming_pool
Pred: bareland, chaparral, lake, snowberg, swimming_pool

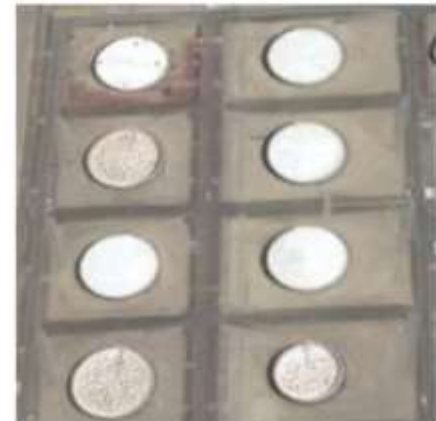


Image 4
True: bareland, lake, parking_lot
Pred: bareland, lake, parking_lot



Image 5
True: bareland, chaparral, cloud, lake, snowberg, swimming_pool
Pred: bareland, chaparral, cloud, lake, snowberg, swimming_pool



Visualizations for Base model for the split ratio 40-10-50.

Image 1
True: chaparral, cloud, golf_course, snowberg
Pred: chaparral, golf_course, snowberg



Image 2
True: chaparral, lake, snowberg, swimming_pool
Pred: bareland, chaparral, lake, snowberg, swimming_pool



Image 3
True: chaparral, cloud, snowberg, swimming_pool
Pred: chaparral, lake, snowberg, swimming_pool



Image 4
True: bareland, lake, parking_lot
Pred: bareland, lake, parking_lot



Image 5
True: bareland, chaparral, cloud, lake, snowberg, swimming_pool
Pred: chaparral, cloud, lake, snowberg, swimming_pool, tennis_court



Experimental Setup

- **For Performance Scheduling**

BATCH_SIZE = 32

initial_learning_rate = 0.005

Epochs = 200

- **With Step Decay**

Initial Learning Rate = 0.005

Decay Steps = 5

Decay Rate = 0.9

Epochs = 200

Comparison between the Step decay model and Performance Scheduling

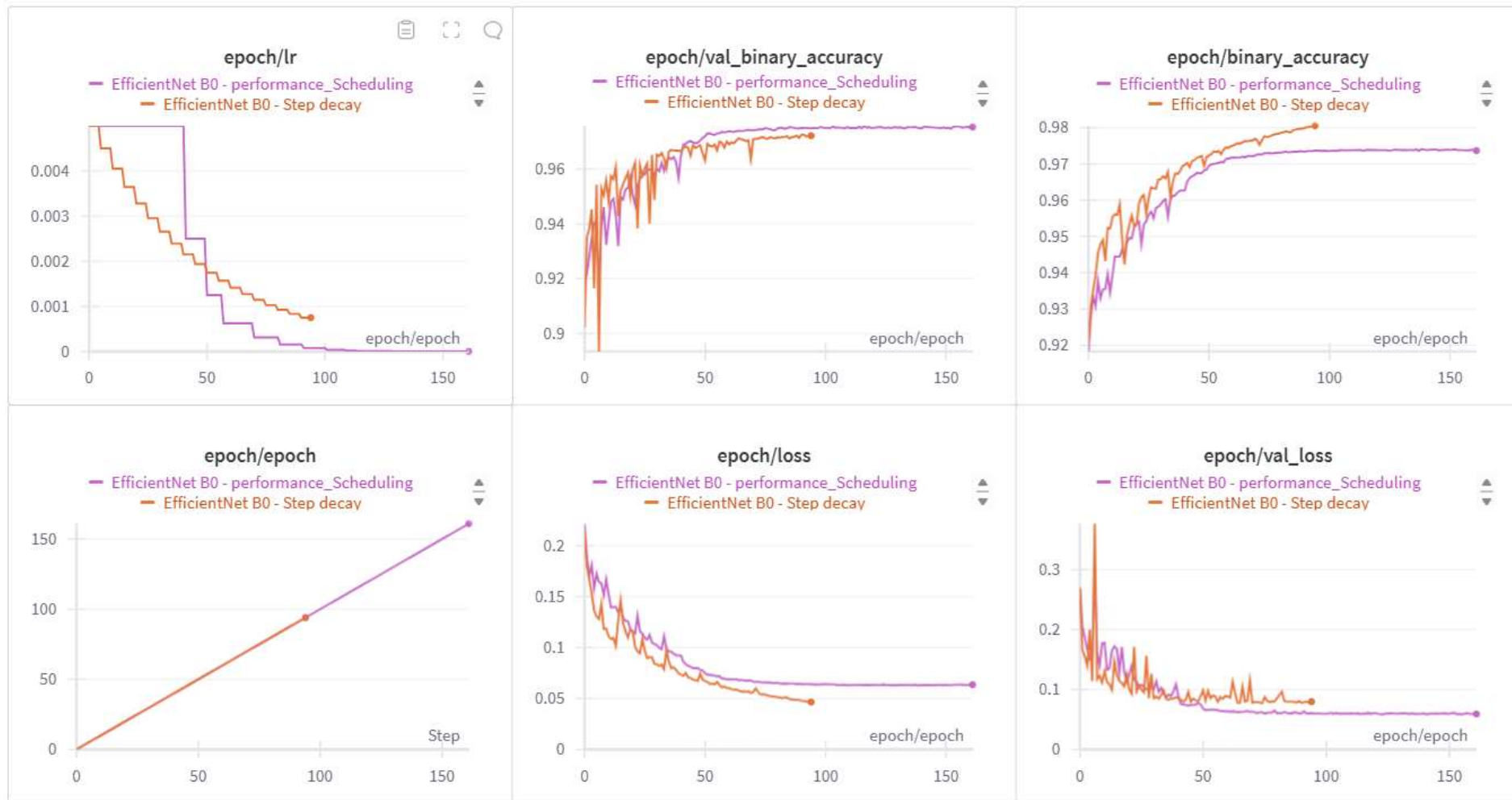


Fig.6. Comparison between the two scratch models

Results for Performance Scheduling

F1 Score (Samples): 0.8568 **Binary Accuracy: 97.58** **ROC AUC Score (samples): 99.22**
Mean Average Precision (mAP): 94.61 **Hamming Loss: 2.41**

Image 1
True: chaparral, cloud, golf_course, snowberg
Pred: chaparral, golf_course, snowberg



Image 2
True: chaparral, lake, snowberg, swimming_pool
Pred: bareland, chaparral, lake, snowberg, swimming_pool



Image 3
True: chaparral, cloud, snowberg, swimming_pool
Pred: bareland, chaparral, snowberg, swimming_pool



Image 4
True: bareland, lake, parking_lot
Pred: bareland, lake, parking_lot



Image 5
True: bareland, chaparral, cloud, lake, snowberg, swimming_pool
Pred: bareland, chaparral, cloud, snowberg, swimming_pool



Results for Step Decay

F1 Score (Samples): 0.8395 **Binary Accuracy:** 97.15 **ROC AUC Score (samples):** 98.93
Mean Average Precision (mAP): 93.53 **Hamming Loss:** 2.84



References and Links

1. [Wandb Report to EfficientNet Base Model B0 for the Split Ratio: 20-10-70.](#)
2. [Wandb Report to EfficientNet Base Model B0 for the Split Ratio: 30-10-60.](#)
3. [Wandb Report to EfficientNet Base Model B0 for the Split Ratio: 40-10-50.](#)
4. [Comparison between Performance Scaling and Step Decay - WandB Report](#)
5. [WandB Report to the EfficientNet Base Model B0 for the Step Decay.](#)
6. [WandB Report for the EfficientNet Base Model B0 for the Performance Scaling.](#)

Thank you for your attention!

Any questions?

[You can have a look at the original paper here!](#)